

Resurrecting Extinct Computers: the Symbolics Lisp Machine

Honour School of Computer Science, Part B

Trinity 2026

Zoe Stafford

Word count: 4994

Abstract

The architecture of modern mainstream processors generally allow for the efficient execution and encoding of common operations in low-level languages, like C. Many high-level languages are implemented using a virtual machine that accepts higher-level instructions, which can reduce the size of compiled programs and simplify the compiler, at the expensive of requiring a complex or slow interpreter to execute. In the 1970s, researchers at MIT developed the *Lisp machine*: a computer designed to efficiently execute Lisp programs by supporting higher-level instructions at the hardware level. This provides the benefits of more compactly encoded programs, faster execution and a simpler compiler at the cost of more complex hardware.

By developing an emulator for a Lisp machine, and writing software for it, this report offers an evaluation of the potential advantages of language-specific architectures. This report documents the Ivory machine, a Lisp machine developed by Symbolics, and outlines the important role the Lisp machines played in the Artificial Intelligence boom of the 1960s-80s and the development of modern computers. For the functions tested, Lisp functions compiled for the Ivory machine require, on average, 25% of the space than when they are compiled for x86_64.

Acknowledgements

I would like to thank my supervisor, Alex Rogers, for his help on this project.

Contents

1	Introduction	4
1.1	Structure of the Report	5
2	Background	6
2.1	Object format	6
2.1.1	Data types	7
2.1.2	Cdr-codes	7
2.2	Instruction set	8
2.2.1	Sequencing	8
2.2.2	Instruction formats	9
2.3	Function calling	10
2.4	Memory	11
2.4.1	Garbage Collection	11
3	An Implementation of an Assembler and Emulator for the Ivory Machine	13
3.1	Assembler	13
3.2	Emulator	15
3.2.1	Memory	15
3.2.2	Executing instructions	15
3.3	Garbage Collection	16
4	Testing and Evaluation	17
4.1	Testing	17
4.2	Evaluation	17
4.2.1	A* search	17
4.2.2	Advantages of the instruction set	20
5	Conclusion	23
5.1	Reflection	23
5.2	Future Work	24
A	Evaluation code	26

Chapter 1

Introduction

Most modern computers use architectures like **x86** and **ARM**, which feature low-level instructions allowing for the efficient execution and encoding of common operations in languages like **C**. This makes compiling high-level languages more complex and often leads to larger compiled programs than if the program were written in **C**. The Lisp machines, on the other hand, featured instructions that perform more complex operations such as taking the **car** or **cdr** of a list. This architecture was designed to allow Lisp programs to be executed faster and to reduce the size of compiled programs. Initially, the Lisp machines succeeded with the first goal: they outperformed general purpose architectures of the time[5]. However, with the huge disparity in investment and research, the Lisp machine's performance advantage and popularity soon shrunk and by the end of 1990s, production of Lisp machines had stopped. This report provides a brief history of the Symbolics Lisp machines and documents some of their distinguishing features, including:

- Tagged architecture: every word contains metadata which allows the hardware to determine the data-type of the object.
- List processing instructions: common operations, including **cdr** and **member** are supported in hardware.
- Hardware-supported garbage collection: the memory barrier and ephemeral objects enable incremental, generational garbage collection, which reduces the length of garbage collection pauses.

The Lisp machines played an important role in the history of computing, especially the development of single-user workstations. Their development lead to many innovations including the Chaosnet networking protocol and influenced the design of many later systems, such as generational and incremental garbage collection. With the current prevalence of garbage-collected, object-oriented languages such as **Python**, it is worth examining the choices the Lisp machines made for efficient execution.

For this project, I developed an assembler and emulator for the Ivory machine in **Rust**. A Lisp implementation of **A*** search is used to evaluate the reduction in program size the Ivory machine offers: Lisp functions compiled for **x86_64** tend to be 4 times larger than for the Ivory machine. The implementation of **A*** search documents some simple techniques that can be used when compiling Lisp for the Ivory machine.

1.1 Structure of the Report

- Chapter 2 provides a brief history of the Lisp machines and introduces the architecture of the Ivory machine.
- Chapter 3 documents the implementation of an assembler and emulator for the Ivory machine.
- Chapter 4 outlines the testing methodology used, and provides a quantitative evaluation of the space savings offered by the Ivory machine's instruction set and a qualitative evaluation of the complexity of compiling to the Ivory machine.

Chapter 2

Background

The Lisp machines are a product of the Artificial Intelligence boom of the 1960s-80s, since Lisp proved to be a popular language for AI research[7] and increasingly complex AI programs required more computation speed and memory than existing computers supported.

Development of Lisp machines began in 1973 at the MIT AI lab, where Tom Knight developed the CONS machine[6] as the processor for MIT's prototype Lisp machine[2]. Symbolics and Lisp Machines Incorporated were later founded out of the MIT AI lab to commercialise the Lisp machines. Symbolics' first Lisp machine was based on the CADR machine, developed at MIT, but their later designs made significant changes. The Ivory machine was the first Lisp machine from Symbolics to feature a single-chip processor.

Fortunately, the Ivory machine's Architecture Specification[4] has been preserved and documents the Ivory machine's instruction set, memory subsystem and the structure of objects.

2.1 Object format

Memory is organised into 40-bit words which can be interpreted as *objects* with 8 bits of metadata and a 32-bit immediate value or a pointer. The hardware uses the metadata to determine the type of each operand, since many instructions accept multiple types of data. This metadata is required for hardware-supported garbage collection as the memory barrier needs to know if a word read from memory is a pointer.

Special purpose hardware allows some runtime type checks to be moved to hardware, which can improve performance. For example, the `add` instruction was implemented by speculatively performing `fixnum` addition while simultaneously checking the operands are `fixnums`.

This section focuses on the interpretation of words as objects; see Section 2.2 for the encoding of instructions. Objects have three fields:

Field	Bits	Usage
<code>cdr-code</code>	38:39	Compact encoding of linked lists
data type	32:37	Describes the immediate field
immediate	0:31	The data or a pointer

The following sections document some of the available data types and corresponding interpretation of the immediate field, and the use of `cdr-codes` for linked lists.

2.1.1 Data types

The data type field describes the data in the immediate field. Some data types are described below.

Immediate data. Each word can store 32 bits of data inline, which is enough for small values including:

- **ntp-fixnum**: a 32-bit signed integer.
- **ntp-small-ratio**: a ratio represented by a 16-bit numerator and denominator.
- **ntp-single-float**: a 32-bit floating point number.

List-like objects. Many objects are represented using a cons-cell or sequence of cons-cells. See Section 2.1.2 for details of the compact representation of linked lists.

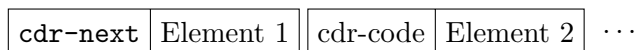
- **ntp-list**: a non-empty linked list. The empty list is represented with **ntp-nil**.
- **ntp-double-float**: a 64-bit floating point number split across the **car** and **cdr** of a cons-cell.
- **big-ratio**: an arbitrary precision ratio.

2.1.2 Cdr-codes

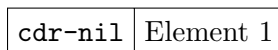
Lisp programs often use linked lists so a compact representation of lists is vital for an efficient Lisp machine. The Ivory machine can encode linked lists with as little as 2 bits per element of overhead.

The empty list is represented by the symbol **nil** and non-empty lists are represented by a pointer to a non-empty sequence of words where the **cdr-code** of each word describes where the next element of the list is.

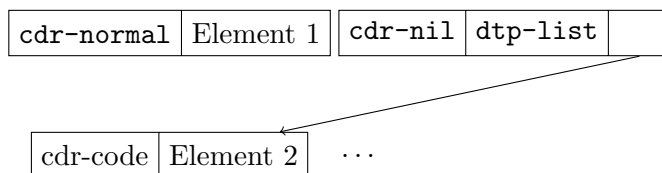
- **cdr-next**: the next word in the sequence is the next element of the list.



- **cdr-nil**: this word is the final element in the list.

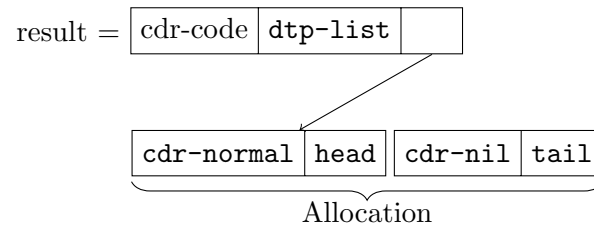


- **cdr-normal**: the next word is a pointer to the next element.

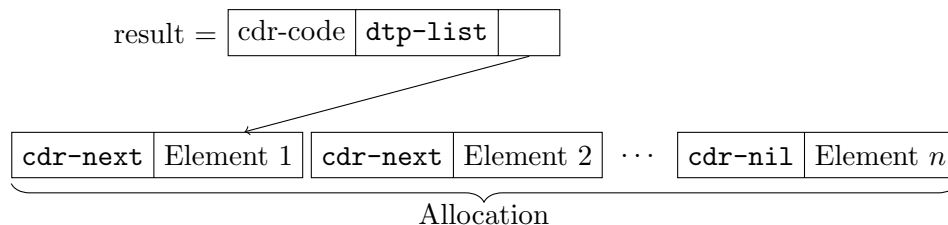


- `cdr-illegal`: this `cdr-code` is not valid in a list and is used in other data structures.

`(cons head tail)` is implemented by allocating a block of 2 words:



A list of length n can be allocated using just n words: half the space required if only `cdr-normal` were supported.



2.2 Instruction set

Many common Lisp operations, such as `car` and `cdr`, are supported directly in hardware, which allows programs to be represented more compactly than on general purpose architectures. Most commonly used operations are represented using just 18 bits. This requires the program counter to store the address of the next instruction as well as which half of the word it is in.

2.2.1 Sequencing

There are 2 classes of instructions: 18-bit *half-word* instructions and 38-bit *full-word* instructions. Full-word instructions are word aligned, so padding is sometimes required after a half-word instruction. To avoid this, the Ivory machine interprets the `cdr-code` field as a selector, called the PC-code, for how to adjust the program counter after each instruction.

- `pc+1`: Add 1 half-word to the program counter.
- `pc-illegal`: The processor traps if it executes a word with this PC-code.
- `pc-1`: Subtract 1 from the program counter.
- `pc+23`: If the current program counter is even then add 2 half-words, otherwise add 3 half-words.

Figure 2.1 shows how PC-codes allow instructions to be encoded without padding, and how PC-code `pc+23` can be used to skip over padding bits, when they are required.

A	half-word	Address	PC-code	even	odd
B	full-word	000	pc+23	A	C
C	half-word	001	pc-1	B	

(a) Avoiding padding

A	half-word	Address	PC-code	even	odd
B	full-word	000	pc+23	A	
C	full-word	001	pc+23	B	
		002	pc+23	C	

(b) Skipping padding bits

Figure 2.1: Instruction sequencing

2.2.2 Instruction formats

Most 18-bit instructions consist of an 8-bit op-code and a 10-bit value, which is often interpreted as a stack offset or an immediate.

Stack offsets. The Ivory machine maintains 3 pointers into the stack: the stack pointer (SP) which points to the highest word in the stack, the frame pointer (FP) which points to the start of the current stack frame, and the local pointer (LP) which locates the `&rest` argument and local variables. Offsets from these pointers can be encoded using 2 bits to identify the register and 8 bits to encode the offset:

- FP and LP can be offset by a non-negative integer, for example `FP|10` or `LP|0`.
- SP can be offset by a non-positive integer, for example `SP|0` and `SP|-5`.

Some instructions support two additional addressing modes:

- `sp-pop` pops a value of the stack.
- An 8-bit immediate which is sign-/zero-extended to a `fixnum`.

If an instruction expects multiple arguments, then the final argument is addressed by the stack offset before the other arguments are popped from the stack. If `sp-pop` is used, then the value for that is popped before other arguments.

The Ivory machine has a top-of-stack register (TOS) which usually holds the value of `SP|0`, except some instructions leave it invalid. To simplify the hardware, some instructions can use TOS instead of reading from the stack, in which case, the compiler must ensure TOS is valid using `movem SP|0`.

Immediate. Some 18-bit instructions interpret the last 10 bits as an integer, or split it into multiple fields. For example, field extraction instructions use 5 bits to encode the field size and 5 bits to encode the location of the bottom bit.

2.3 Function calling

Function calling is split into three parts: starting the call, pushing the arguments to the stack, and finishing the call. The following registers are used during function calls:

- Continuation register: Normally this stores the return address; between the **start-call** and **finish-call** instructions this stores the address of the function about to be called.
- Control register: This contains fields including the argument count, the size of the caller's frame, and the value-disposition for this call.
- Stack pointer, local pointer and frame pointer

Starting a function call. The Continuation and Control registers are saved to the stack, then the Continuation register is set to the address of the callee and some flags are set in the Control register. For some functions, including generic functions and closures, an extra argument is pushed. There are multiple instructions to start a function call, depending on the type of the function and if the address is known at compile time:

- **call-compiled**: call a non-generic function at a known address.
- **start-call** *arg*: call a function specified by *arg*.

Pushing arguments. Once the function call has been started, arguments are pushed to the stack. In this stage, only FP and LP relative addressing should be used, as the **start-call** instruction pushes a variable number of values.

Finishing a function call. The **finish-call** instruction requires the number of arguments, which can be encoded in the instruction or read from the top of the stack. FP is set to the location that the Continuation register was saved and LP is set one past the last supplied argument. The Continuation register is set to the address of the next instruction then the Control register is updated and the program counter is set to the address of the callee. Finally, the processor checks that the stack cache contains the stack and if not, a stack-overflow trap occurs to refill the stack cache. This instruction sets the value-disposition, which determines what is done with the returned values:

Effect Discard the returned values.

Value Push the first returned value to the stack, or **nil** if no values were returned.

Multiple Push all the returned values followed by the number of returned values.

Return Immediately re-execute the return instruction, so the values returned from the callee are then returned from the caller.

Managing arguments and local variables. To allow functions to handle optional arguments, there is an `entry` instruction, which checks that the correct number of arguments have been provided and pushes the default value for any uninitialised optional arguments.

In the simplest case, without a `&rest` parameter or `apply` argument, this instruction checks there are at least as many arguments as required parameters and no more than the number of required and optional parameters, then jumps into the *entry vector*, which contains instructions to push the default value of any optional parameters.

Arguments can be pushed to or pulled from the `apply` argument to fill the required and optional parameters and any remaining arguments form the `&rest` argument, if the function accepts one. Arguments start at `FP|2` and local variables are stored after the arguments, except in functions that accept a `&rest` argument, where the `&rest` argument is located at `LP|0` and local variables start at `LP|1`.

Returning from a function. The Continuation register is read to determine the return address before being restored from the stack. The callee’s stack frame is removed, then `FP` is restored using the frame size in the saved Control register. The Control register and `LP` are restored then the returned values are pushed. There are 3 `return` instructions:

- **return-single:** return the top of the stack, `nil` or `true`
- **return-multiple:** return multiple values from the stack, with the count given by an immediate value or popped from the top of the stack
- **return-kludge:** ignore the value-disposition and just push the returned values to the stack. This is used by trap handlers to implement instruction that aren’t supported directly in hardware.

2.4 Memory

The address space is split into 32 zones, the first 31 of which are in the virtual address space. Addresses in these zones are translated using the page hash table (PHT) to find the physical address and check metadata and permissions. The `lookup` operation is implemented in microcode, with a hardware cache of recently used pages. Zone 0 is the ephemeral address space, zones 1-30 are general purpose virtual memory and zone 31 is unmapped to provide direct access to physical memory.

2.4.1 Garbage Collection

The Ivory machine uses a copying garbage collector[8], which includes four techniques to decrease the length of garbage collection pauses and improve the efficiency of the garbage collector:

Incremental garbage collection The hardware intersperses execution of program instructions and tracing live values.

Approximately depth-first copying Objects are copied approximately depth-first, improving the spatial locality of data.

Ephemeral address space Many objects in Lisp have short lifetimes, so they can be collected more frequently without having to scan the entire address space—this requires the garbage collector to track where references to ephemeral objects might be.

Hardware assistance Special purpose hardware improves the efficiency of garbage collection, for example a memory barrier ensures the stack and registers can't contain references to old-space.

Garbage collection is managed by three components:

- The mutator: Executes program instructions and allocates new objects.
- The transporter: Copies values from old-space to copy-space.
- The scavenger: Scans old-space for live objects then invokes the transporter.

There are three types of virtual memory pages:

- New-space: Where new objects are allocated.
- Old-space: A page that is waiting to be copied.
- Copy-space: Where objects from old-space are copied.

When a word containing a reference to old-space is read from memory, the transporter copies the referenced object to copy-space and updates the word being read. This memory barrier ensures that in normal operation, the stack and registers only contain addresses in new- or copy-space, which ensures that every object has a canonical identity and that new-space can't contain references to old-space so the scavenger can skip scanning new-space.

The scavenger traces from the roots, including the stack and registers, to find and copy all reachable objects. When the mutator allocates an object, the scavenger is run for time proportional to the size of the allocation, which ensures that by the time new-space fills up, old-space has been evacuated and can be reused.

To approximate depth-first copying, the scavenger adapts the Cheney algorithm[1] as follows:

- If there is a partially filled and unscanned page at the end of copy-space, then scan that page.
- Otherwise, scan from the start of copy-space.

This generally ensures that the children of an object are copied soon after the object itself is, but without further bookkeeping requires some pages to be scanned twice. The second scan through a page is cheap, since the objects have already been transported.

Chapter 3

An Implementation of an Assembler and Emulator for the Ivory Machine

As some details are missing from the available documentation, some assumptions have been made. Generally, the emulator implements each instruction as described in the Architecture Specification, with some exceptions described in Section 3.2. The emulator and assembler are implemented in Rust. Rust was chosen as it features algebraic data types to conveniently represent instructions and allows writing low-level code, when required for performance, while still including many high-level features.

3.1 Assembler

The assembler and emulator share a representation of instructions:

```
pub enum MacroOp {
    List(List),
    Predicate(Predicate),
    Numeric(Numeric),
    DataMove(DataMove),
    FieldExt(FieldExt),
    Array(Array),
    Branch(Branch),
    Block(Block),
    FnCall(FnCall),
    Bind(Bind),
    Catch(Catch),
    LexicalVar(LexicalVar),
    InstanceVar(InstanceVar),
    Subprimitive(Subprimitive),
    Constant(Constant),
    Other(Other),
    /// A directive used by the assembler.
    Directive(Directive),
}
```

```

pub enum FnCall {
    StartCall { offset: StackOffset },
    CallCompiled { address: LabelOr<HalfWordAddress>, prefetch: bool },
    CallIndirect { address: LabelOr<VAddress>, prefetch: bool },
    CallGeneric { address: LabelOr<VAddress>, prefetch: bool },
    FinishCallN { args: u8, apply: bool, disposition: Disposition },
    ReturnSingle(ReturnSingleArg),
    ReturnMultiple(StackOffset),
    ReturnKludge(StackOffset),
    Entry { rest_accepted: bool, max_args: u8, min_args: u8 },
}

```

Due to the complexity of instruction sequencing, described in Section 2.2.1, the assembler is implemented as a state machine, which allows encoding a single instruction at a time:

```

enum PackingStatus {
    Even { address: VAddress },
    Odd { address: VAddress },
    EvenGap { address: VAddress },
}

```

- `Even { address }`: `address` is empty. Any instruction can be placed at `address`.
- `Odd { address }`: `address` contains a even half-word instruction. A half-word instruction can be placed at `address` odd or a full-word instruction can be placed at `address + 1`.
- `EvenGap { address }`: `address` contains an even half-word instruction, `address + 1` is full and `address + 2` is empty. A half-word instruction can be placed at `address` odd or a full-word instruction can be placed at `address + 2`.

The assembler resolves labels while encoding instructions in a single pass by maintaining 2 maps:

- The first maps from labels to the address the corresponding instruction is encoded at.
- The second maps labels to instructions that need re-encoding, since they occur before the label they reference.

When the assembler encodes a labelled instruction, the label and address are added to the first map and any instructions referencing this label in the second map are re-encoded.

The Ivory machine came with an extensive standard library[9], which Symbolics kept proprietary, so a part has been re-implemented. To support a standard library, the `include_preprocessor` crate¹ is used to allow C-style `#include` directives. The assembly file `stdlib.s` contains some standard functions, such as `cons` and `equal`, and the entry-point, which sets up the stack before calling the user-defined `main` function.

The assembler supports two directives:

¹<https://crates.io/crates/include-preprocessor>

- **end-of-function-fence**, which emits a word with PC-code `pc-illegal`. This directs the instruction cache to stop prefetching instructions after this point.
- **@no-pack**, which tells the assembler to align the instruction to a word boundary. This is useful in function entry vectors.

3.2 Emulator

The state of the emulator is stored in a `struct` containing the value of registers, main memory and some state required while executing an instruction.

3.2.1 Memory

Physical memory. Physical memory with a 32-bit address space is implemented by allocating 2^{32} zeroed words, which Linux supports efficiently using copy-on-write pages. Due to how Rust implements large zeroed allocations, using `Vec<Word>` writes 0 to each word, rather than allowing the OS to allocate zeroed pages, so `Vec<u8>` is used to allocate memory and then cast to an array of `Words`.

Virtual memory. The Architecture Specification documents the format of the PHT and provides a `lookup` procedure in Lisp, which is straightforwardly translated to Rust. The procedure to add an entry to the PHT was not documented, but can be done by:

- Using a loop similar to the one in `lookup`, locate the first invalid entry in the collision chain.
- Replace the invalid entry with the new entry.
- Increment the collision count of each previous bucket in the collision chain.

3.2.2 Executing instructions

Unlike the original machine, the emulator is not pipelined, although the execution of instructions is split into fetch, decode and execute steps:

- The fetch step reads a word from memory.
- The decode step uses the program counter to decode the relevant part of the fetched word and computes the next program counter, according to the PC-code of the fetched word.
- The execute step does the rest: reading and writing from memory, computing any results and updating registers. Since instructions can perform complex operations, it is not feasible to break down this step any further.

List operations. List operations are a good example of how instructions are implemented, for example the `car` instruction is implemented by:

```
let obj = self
    .read_stack_offset(op.offset, ImmediateMode::ZeroExtend)?
    .object();
let car = self.car(obj)?.with_cdr_code(CdrCode::Next);
self.push(car.into(), true)?;
```

The `read_stack_offset` method loads the operand from the stack, which is then interpreted as an object. The `car` method computes the head of the list by dispatching on the data type of `obj` and using the `read_address` method to fetch the `car`.

Some instructions, including `member` and `assoc`, are *interruptible* meaning that traps can occur while they are executing causing them to restart. When applying these operations to very long lists, interrupts can occur often enough that these instructions never complete; to avoid this, these instructions are also implemented in assembly so that further interrupts don't restart the instruction. The emulator only includes an implementation in assembly, which is called when the respective instruction is executed.

3.3 Garbage Collection

While a high level description of the garbage collection algorithm used on the older Symbolics 3600 is provided[8], the documentation does not provide exact implementation details.

The garbage collector tracks which physical pages are currently in use by dividing physical memory into 256-page chunks; the first page is metadata about which of the other pages are currently in use. The metadata page stores the number of free pages and a contiguous list of the physical address of each free page.

The garbage collector tracks which virtual pages are old-space, new-space and copy-space. The division between old-space and new-/copy-space is maintained by the Zone old-space register, which allows the memory barrier to trigger a transport trap when reading a word that addresses old-space.

The garbage collector maintains the following metadata about each zone:

- The address of the next free word in this zone.
- Two pointers used by the modified Cheney algorithm: the next address from the start to scan and the next address in the partially filled page to scan.

The scavenger scans the stack by reading the Continuation register from `FP|0` to detect the start of the stack and the Control register from `FP|1` to determine the size of the next stack frame. After scavenging the stack, the memory barrier guarantees that no references to old-space can be written to the stack, so the scavenger only needs to scan copy-space.

Chapter 4

Testing and Evaluation

4.1 Testing

Individual components of the emulator were tested using Rust’s unit testing framework and the `proptest` crate to perform randomised property tests. These property tests proved useful in finding bugs during the development of the emulator, as randomised testing can more effectively test edge cases.

The emulator was also tested with integration tests, which are organised as a file containing the assembly to execute, and a description of the expected results in Rust. For example, Figure 4.1 shows how the `cons` function and `car` and `cdr` instructions can be tested.

Each integration test is executed as follows:

- Assemble the test program.
- Initialise the emulator with BAR3 set to a location in physical memory.
- Execute the program.
- Check the program stopped for the expected reason.
- Check the program has written the expected values to BAR3.

4.2 Evaluation

Two of the proposed benefits of the Lisp machines are that compiled programs are often smaller and that specialised hardware can improve efficiency. The Ivory machine was micro-coded but the emulator is not, so it is hard to compare efficiency between the Ivory machine and contemporary general purpose computers.

4.2.1 A* search

To evaluate the reduction in program size, A* search[3] was implemented in Common Lisp (see Figure 4.2) and compiled by hand into Ivory machine assembly and by the Clozure Common

```

#include <stdlib.h>

.main:
    ; (cons 10 nil)
    call-compiled .cons
    dtp-fixnum 10
    constant nil
    finish-call-n 2 value

    ; Check car and cdr are correct
    car SP|0
    %block-3-write sp-pop
    cdr SP|0
    %block-3-write sp-pop

    return-single nil
    end-of-function-fence

```

(a) File tests/cons.s

```

TestProgram::new("tests/cons.s")
    .expect_halt()
    .expect_result(Object::fixnum(10))
    .expect_result(Object::nil())

```

(b) Test description

Figure 4.1: Integration test for `cons`, `car` and `cdr`

```

(defun a-star-search (h neighbours weight start goal)
  (do ((frontier (acons start 0 nil))
      (parent nil)
      (cost-so-far (acons start 0 nil)))
      ((null frontier) nil)

      (let ((current (car (pop frontier))))
        (if (equal current goal)
            (return (compute-path parent goal)))

        (loop for next in (funcall neighbours current) do
          (let ((new-cost
                (+ (cdr (assoc-equal current cost-so-far))
                   (funcall weight current next)))
              (old-cost (assoc-equal next cost-so-far)))
            (if (or (null old-cost) (< new-cost (cdr old-cost)))
                (block nil
                  (setq cost-so-far (acons next new-cost cost-so-far))
                  (setq frontier
                    (queue-insert
                     next
                     (+ new-cost (funcall h next))
                     frontier))
                  (setq parent (acons next current parent))))))))))

```

Figure 4.2: Implementation of A* search in Lisp

Function	x86_64 (bytes)	Ivory machine (bytes)	Difference
<code>member</code>	156	40	26%
<code>queue-insert</code>	273	65	24%
<code>compute-path</code>	274	60	22%
<code>a-star-search</code>	892	240	27%

Figure 4.3: Size of compiled functions compiled on `x86_64` and on the Ivory machine

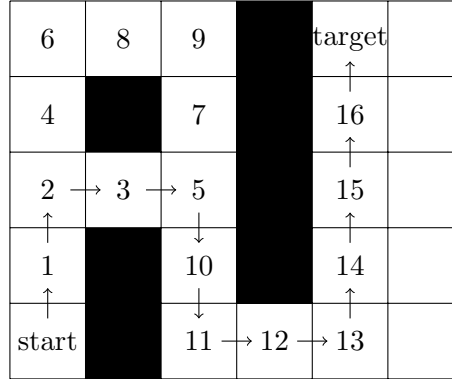


Figure 4.4: Grid used to test A* search

Lisp compiler¹ (`cc1`) into `x86_64` assembly. The A* search implementation makes extensive use of lists to implement priority queues and maps.

List processing instructions, such as `car` and `endp`, simplify the assembly significantly. On a general purpose architecture these operations would require multiple instructions, including branches depending on the data type of the arguments. Figure 4.3 shows the compiled size of functions implementing A* search on `x86_64` and for the Ivory machine. On average, the implementation in Ivory machine assembly takes 25% of the space as on `x86_64`. Since this implementation makes extensive use of list processing instructions, the space savings may not be so large for other workloads. The implementations of `compute-path` are included in the next section and the rest of implementations can be found in Appendix A.

The implementations of A* search were tested on a 2D grid using the taxi-cab distance as the heuristic. The test grid used is shown in Figure 4.4: white squares indicate nodes and each square is connected to the adjacent white squares. Both implementations were extended to log nodes in the order they are searched, shown as the numbering of each square. The shortest path is indicated with arrows. A* search first explores squares that reduce the heuristic distance to the target, which allows it to explore less of the state space than Dijkstra’s algorithm in this case.

4.2.2 Advantages of the instruction set

This section documents how some Lisp operations can be implemented on the Ivory machine to provide a qualitative evaluation of how the architecture of the Ivory machine could simplify

¹Version 1.13 (<https://cc1.closure.com/>)

writing a Lisp compiler for the Ivory machine and to explain how the space savings documented in the previous section are achieved.

The `pop` macro, which removes the `car` of a list and returns it, can be implemented on the Lisp machine using the `set-to-cdr-push-car` instruction, whereas on `x86_64`, `cc1` compiles this to 11 instructions, including a branch.

The `set-to-cdr-push-car` instruction is also useful to implement loops of the form `(loop for el in list do body)`:

```
.loop-body:
    set-to-cdr-push-car SP[0]
    ; body
.loop-cond:
    endp SP[0]
    branch-true .loop-exit
```

Idiomatic Lisp code often uses higher-order functions, which is straightforward to implement with the `start-call` instruction, since it handles generic functions, closures and interpreted functions. The `entry` instruction simplifies the process of accepting optional and `&rest` arguments.

Figure 4.5 shows the implementation in Ivory machine assembly of `compute-path`, which constructs the path from the target to the source using the `parent` association list.

```

(defun compute-path (parent node)
  (do ((path nil))
      nil
      (setq path (cons node path))
      (let ((p (assoc-equal node parent)))
        (if (null p) (return path)
            (setq node (cdr p))))))

```

(a) Lisp implementation

```

.compute-path:
  ; FP/2 = parent, FP/3 = node
  entry-rest-not-accepted 2! 0?

  ; FP/4 = path = nil
  constant nil
.compute-path-loop-start:
  ; (setq path (cons node path))
  call-compiled .cons
  push FP|3
  push FP|4
  finish-call-2 value
  pop FP|4

  ; FP/5 = p = (assoc-equal node parent)
  call-compiled .assoc-equal
  push FP|3
  push FP|2
  finish-call-2 value

  ; (if (null p) (return path))
  branch-false-else-no-pop .compute-path-return

  ; (setq node (cdr p))
  set-to-cdr SP|0
  pop FP|3

  branch .compute-path-loop-start
.compute-path-return:
  ; (return path)
  push FP|4
  return-single tos
end-of-function-fence

```

(b) Assembly implementation

Figure 4.5: Implementations of `compute-path` in Lisp and assembly

Chapter 5

Conclusion

The main contributions of this project are an emulator and assembler for the Ivory machine and an introduction to the architecture of the Ivory machine. Implementing and evaluating this architecture has shown that Lisp machines in the 1980s achieved their goal of increasing the size and complexity of AI programs that could be run.

However, with the onset of the second AI winter, research into AI, and with it the Lisp machines, all but ceased. It seems unlikely that language specific architectures will ever have a large market share, due to the relatively small investment in their development compared to general purpose architectures.

Additionally, two of the main advantages of the Lisp machines (speed and compiled program size) are much less relevant given that programs for high performance computing are often written in low level languages and that storage is much larger and more affordable than in the 1980s.

Lisp's relatively small market share contributed to the decline of the Lisp machines, however many Lisp machines could also run programs written in Prolog. In fact, the CONS machine was designed to allow running different macro-architectures by allowing the decoding of macro-instructions into micro-ops to be reprogrammed[6].

5.1 Reflection

Researching the Ivory machine proved difficult, since many details of the machine were never made public. In particular, I was unable to find many details about the implementation of hardware-supported garbage collection, which is one of the defining features of the Lisp machines, so I had to rely on the little information available about garbage collection on the Symbolics 3600.

Initially work on the emulator was challenging, since many components of the emulator depend on each other and some components, especially the virtual memory map, are complex. I found the `tracing` crate invaluable for providing logging with useful context, even in deeply nested functions.

Once I implemented memory, the object format and instruction decoding, implementing each instruction was relatively straightforward. The Architecture Reference categorises instructions by their semantics rather than their encoding, which slightly complicated decoding instructions, but made it easier to encapsulate operations common to each category of

instructions.

5.2 Future Work

Without access to an Ivory machine, it was not possible to compare the emulator to real hardware. Access to an Ivory machine could allow more thorough documentation of the implementation of garbage collection, as well as a quantitative evaluation of the performance advantage offered by the Ivory machine over contemporary general purpose computers.

There is little publicly available information about the micro-architecture of the Ivory machine, so access to an Ivory machine may allow the micro-architecture to be reverse engineered. Without access to an Ivory machine, designing a micro-architecture for Ivory machine assembly could determine to what extent the complex instructions of the Ivory machine increase the complexity of designing hardware.

To determine how effectively other high-level languages could be executed by reprogramming the macro-architecture, an instruction set, such as CPython bytecode, could be implemented in CONS machine micro-code.

Bibliography

- [1] C. J. Cheney. “A nonrecursive list compacting algorithm”. In: *Commun. ACM* 13.11 (Nov. 1970), pp. 677–678. ISSN: 0001-0782. DOI: 10.1145/362790.362798. URL: <https://doi.org/10.1145/362790.362798>.
- [2] Richard Greenblatt. *The LISP machine*. Memo. MIT, 1974.
- [3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. In: *IEEE Transactions on Systems Science and Cybernetics* 4.2 (1968), pp. 100–107. DOI: 10.1109/TSSC.1968.300136.
- [4] *I-Machine Architecture Specification*. Available at https://bitsavers.org/pdf/symbolics/I_Machine/I-Machine_Architecture_Specification.pdf. Symbolics. Dec. 1986.
- [5] Cameron Kaiser. *Refurb weekend: the Symbolics MacIvory Lisp machine I have hated*. <https://oldvcr.blogspot.com/2024/10/refurb-weekend-symbolics-macivory-lisp.html>. Accessed: 2026-03-31. Oct. 2024.
- [6] Thomas Knight. *The CONS microprocessor*. Memo. MIT, 1975.
- [7] John McCarthy. “History of LISP”. In: *History of Programming Languages*. New York, NY, USA: Association for Computing Machinery, 1978, pp. 173–185. ISBN: 0127450408. URL: <https://doi.org/10.1145/800025.1198360>.
- [8] David A. Moon. “Garbage collection in a large LISP system”. In: *Proceedings of the 1984 ACM Symposium on LISP and Functional Programming*. LFP ’84. Austin, Texas, USA: Association for Computing Machinery, 1984, pp. 235–246. ISBN: 0897911423. DOI: 10.1145/800055.802040. URL: <https://doi.org/10.1145/800055.802040>.
- [9] *Symbolics Common Lisp: Language Dictionary*. Available at https://bitsavers.org/pdf/symbolics/software/genera_7/999019_Symbolics_Common_Lisp_Language_Dictionary_Aug86.pdf. Symbolics. Aug. 1986.

Appendix A

Evaluation code

This appendix includes the Lisp code and Ivory machine assembly used to evaluate the size of compiled functions on the Ivory machine and x86_64 in Section 4.2.

member.

```
(defun member (item list)
  (loop for el in list do (if (equal item el) (return t))))
```

```
.member:
  entry-rest-not-accepted 2! 0?

  ; FP/4 = tail of list still to check
  push FP|3
  branch .member-loop-cond

.member-loop-start:
  ; (equal (car FP/4) key)
  call-compiled .equal
  car FP|4
  push FP|2
  finish-call-2 value
  branch-false .member-loop-end

  ; Found key in list
  return-single t

.member-loop-end:
  set-to-cdr FP|4

.member-loop-cond:
  branch-true-and-no-pop .member-loop-start

  return-single nil
  end-of-function-fence
```

queue-insert.

```
(defun queue-insert (value cost queue)
  (if (or (null queue) (< cost (cdar queue)))
      (acons value cost queue)
      (cons (car queue)
            (queue-insert value cost (cdr queue)))))
```

.queue-insert:

```
entry-rest-not-accepted 3! 0?

; if (or (null queue) (< cost (cdar queue)))
endp FP|4
branch-true .queue-insert-then

car FP|4
set-to-cdr SP|0
greaterp FP|3
branch-true .queue-insert-then

; else (cons (car queue)
;           (queue-insert value cost (cdr queue)))
call-compiled .cons
car FP|4

call-compiled .queue-insert
push FP|2
push FP|3
cdr FP|4
finish-call-3 value

finish-call-2 return

; then (acons value cost queue)
.queue-insert-then:
call-compiled .acons
push FP|2
push FP|3
push FP|4
finish-call-3 return

end-of-function-fence
```

a-star-search

```
(defun a-star-search (h neighbours weight start goal)
  (do ((frontier (acons start 0 nil))
```

```

    (parent nil)
    (cost-so-far (acons start 0 nil)))
  ((null frontier) nil)

  (let ((current (car (pop frontier))))
    (if (equal current goal)
        (return (compute-path parent goal)))

    (loop for next in (funcall neighbours current) do
      (let ((new-cost
              (+ (cdr (assoc-equal current cost-so-far))
                 (funcall weight current next)))
            (old-cost (assoc-equal next cost-so-far)))
        (if (or (null old-cost) (< new-cost (cdr old-cost)))
            (block nil
              (setq cost-so-far
                    (acons next new-cost cost-so-far))
              (setq frontier
                    (queue-insert
                     next
                     (+ new-cost (funcall h next))
                     frontier))
              (setq parent
                    (acons next current parent))))))))))

.a-star-search:
  entry-rest-not-accepted 5! 0?

  ; FP/7 = frontier
  call-compiled .acons
  push FP 5
  dtp-fixnum 0
  constant nil
  finish-call-3 value

  ; FP/8 = parent
  constant nil

  ; FP/9 = cost-so-far
  call-compiled .acons
  push FP 5
  dtp-fixnum 0
  constant nil
  finish-call-3 value

.a-star-search-do-cond: ; SP = FP/9
  set-sp-to-address FP 9

```

```

endp FP|7
branch-false .a-star-search-do-body

return-single nil

.a-star-search-do-body:
; FP/10 = current = (car (pop frontier))
set-to-cdr-push-car FP|7
set-to-car SP|0

; (if (equal current goal)
;     (return (compute-path parent goal)))
call-compiled .equal
push FP|10
push FP|6
finish-call-2 value
branch-false .a-star-search-not-goal

call-compiled .compute-path
push FP|8
push FP|6
finish-call-2 return

.a-star-search-not-goal:

; FP/11 = (funcall neighbours current)
start-call FP|3
push FP|10
finish-call-1 value

; (loop for next in FP/11 do ...)
.a-star-search-loop-cond: ; SP = FP/11
set-sp-to-address FP|11
endp FP|11
branch-true .a-star-search-do-cond

; FP/12 = next = (pop FP/11)
set-to-cdr-push-car FP|11

; FP/13 = new-cost
; (cdr (assoc-equal current cost-so-far))
call-compiled .assoc-equal
push FP|10
push FP|9
finish-call-2 value
set-to-cdr SP|0

```

```

; (funcall weight current next)
start-call FP|4
push FP|10
push FP|12
finish-call-2 value
add sp-pop

; FP/14 = old-cost
call-compiled .assoc-equal
push FP|12
push FP|9
finish-call-2 value

; if (or (null old-cost) (< new-cost (cdr old-cost)))
; (null old-cost)
endp FP|14
branch-true .a-star-search-then
; (< new-cost (cdr old-cost))
cdr FP|14
greaterp FP|13
branch-true .a-star-search-then

branch .a-star-search-loop-cond

.a-star-search-then:
; (setq cost-so-far (acons next new-cost cost-so-far))
call-compiled .acons
push FP|12
push FP|13
push FP|9
finish-call-3 value
pop FP|9

; (setq frontier (queue-insert ...))
call-compiled .queue-insert
; next
push FP|12
; (+ new-cost (funcall h next))
start-call FP|2
push FP|12
finish-call-1 value
add FP|13
; frontier
push FP|7
finish-call-3 value
pop FP|7

```

```
; (setq parent (acons next current parent))
call-compiled .acons
push FP|12
push FP|10
push FP|8
finish-call-3 value
pop FP|8

branch .a-star-search-loop-cond
end-of-function-fence
```